

TEAM WORKING DOCUMENT | 09 OCTOBER 2026

EV Bike Battery MVP

Complete Project Plan and Team Execution Guide

Project detail	Working definition
Product	Find a verified compatible EV bike battery, buy it with installation, select a service slot, and track fulfillment.
Platforms	Responsive website plus a real Android/iOS mobile client, backed by one shared API and data source.
Team	12 contributors with one accountable owner and one separate feature branch per person.
Plan	Eight-week baseline. Re-estimate at weekly reviews; phase exits depend on evidence, not dates alone.
Repository snapshot	Remote main 88cb88d; ui acd4855; logistics 1d486e1. Local checkout is currently on logistics.

1. Executive Summary

Build the smallest trustworthy service that lets an EV bike owner identify a compatible battery, understand the complete price, book installation, and follow the order to completion. The website and mobile app should share the same backend rules, fitment data, quote, order, appointment, and status records.

The repository already contains a substantial responsive web prototype with customer, vendor, and technician views. Its visible data and actions are driven by browser-side fixtures. Treat it as a reusable interaction prototype, not proof that authentication, price calculation, slot booking, inventory, dispatch, or tracking are production-ready.

Recommended product structure: keep one repository. Continue the existing website in `web/`, build the installable phone client in `mobile/`, add the server in `backend/`, and keep the versioned OpenAPI/data contract in `contracts/`. The separate clients call the same server; do not fork pricing or fitment rules into each client.

Project principle

Compatibility must come from reviewed catalog data. A server-generated quote is the source of truth. Every role can perform only authorized actions. If fitment, stock, service coverage, or a slot is unknown or unavailable, explain that clearly and provide a next step.

2. Current Project State and Branch Audit

The following snapshot comes from the remote Git heads checked on 09 October 2026. The file trees were compared across all three branches; the DOCX supplied with this request was used only for visual style.

Branch	Observed state	Action for the team
main 88cb88d	Current remote default head. Contains the web prototype and includes the ui branch changes in its history.	Use updated origin/main as the agreed feature base after confirming the maintainer's merge policy.
ui acd4855	UI alignment and branding commit. It is already an ancestor of main, so its changes are in main.	Do not re-merge or create a personal branch from this stale head.
logistics 1d486e1	Adds logistics/README.md. The branch also carries older web styling/content than current main.	Review the logistics document as a standalone contribution. Port or PR the relevant requirements; do not merge the whole branch blindly.
Local checkout	Currently on logistics. Local main is at d27f5c9, behind origin/main 88cb88d.	Fetch and start personal work from origin/main, not the stale local main or logistics checkout.

Implemented prototype work

Area	Present in the inspected web tree
Web structure	Vite-served web/ app with vanilla JavaScript and CSS tokens/components; customer, vendor and technician views.
Customer journey	Bike and battery discovery, product details, booking flow, order views and simulated tracking surfaces.
Operations prototype	Vendor order/stock/slot controls and technician job/checklist/status interactions.
Shared UI widgets	Map visualization and digital signature components; responsive preview toolbar.
Seed data	Browser store and mock fixtures for bikes, batteries, fitment, service centers, prices, slots, technicians and orders.

Still to build: no actual mobile project, backend/API server, database/migrations, production authentication, payment integration, authoritative slot reservation, persistent rider assignment, or real GPS integration is present in the inspected branches. UI states that appear functional today are primarily local mock behavior.

Existing logistics document needs scope alignment

The logistics branch document proposes a broad e-commerce/logistics platform, advanced route optimization, MCP operations, and quantum/hybrid solving. These are not prerequisites for the battery-installation MVP. Keep the MVP centered on EV bike batteries and installation; record any broader product or research work as deferred unless the product owner changes scope.

3. Product Goal, Users, and MVP Scope

The customer must be able to identify a battery that fits a specified bike, see stock and seller information, review the complete charge before confirming, select an installation option and slot, and follow fulfillment. The vendor and provider tools support that journey; they are not separate products for the first release.

Role	Primary MVP need	Main surfaces
Customer	Find a fitment-backed battery, see total cost, book installation, and track the order.	Sign-in, bike profile, catalog/search/detail, quote/checkout, booking, order history/status, help.
Seller	Manage listings, verified fitment, price, stock, and incoming order response.	Seller dashboard, products, inventory, incoming orders, accept/reject with reason.
Rider / technician	See assigned work and appointment details; accept or decline; update permitted milestones.	Assigned jobs, job details, appointment/address, permitted status actions, checklist.
Admin / support	Resolve access, seller/provider, order, coverage, and service issues.	Minimum internal review and issue-resolution tools; exact admin screen scope set in Week 1.

Included in the first release

- Customer account and bike profile with make, model, year and variant; VIN is optional only if the team verifies a reliable lookup source.
- Battery catalog with specification, seller, price, stock, warranty, compatibility evidence and installation availability.
- Search, filter, product detail, cart/order, authoritative quote, checkout, appointment selection, confirmation, order history and status.
- Seller product, stock and order-response workflows; rider/technician assigned-job and status workflows; basic admin/support operations.
- Delivery and installation fees and estimates from explicit rules. Status timestamps are sufficient for MVP tracking.

Defer unless approved

General bicycle products, broad workshop management, multi-vendor checkout, subscriptions, financing, loyalty, route optimization, live GPS maps, voice search, and quantum algorithms in production checkout, compatibility, dispatch, or scheduling. AI must not decide compatibility without verified fitment records.

Trust rules

Situation	System behavior
Verified fitment	Return compatible only when a reviewed record supports the bike make/model/year/variant and required electrical/physical constraints.
Unknown fitment	Show unknown and explain what bike detail or catalog review is missing; never convert missing data into a match.
Stock or slot changes	Recheck on the server at order confirmation and show a recovery path when no longer available.
Price	Show every approved line and the final total before confirmation. Changed totals require customer confirmation.
Provider contact	Reveal after authorized assignment only; enforce on the server.

4. Website and Mobile App Architecture

One repository is recommended for this team. Keep web and mobile source code separate and share a single authenticated API, data contract, catalog, quote/order state, and role rules. This supports a coordinated release without forcing the web prototype to be rewritten as mobile UI.

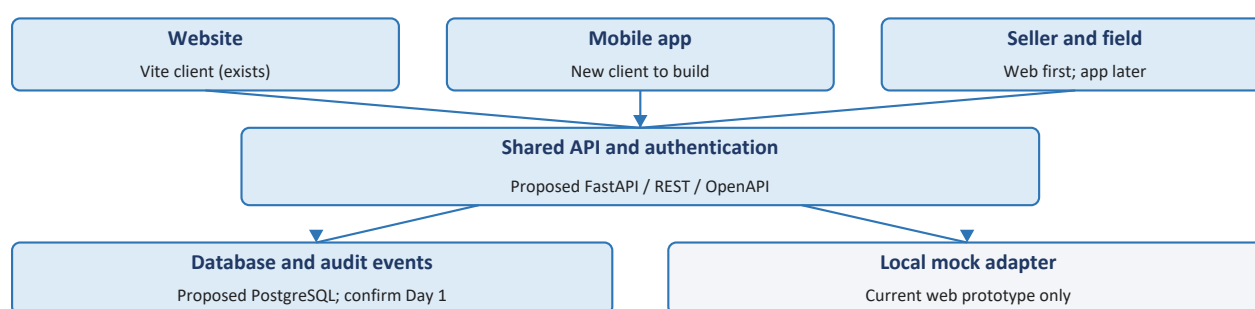


Figure 1. Proposed client, API, and persistence boundary. Only the responsive web client and local mock adapter exist today.

Repository area	Purpose and owner	Current status
web/	Responsive website and operations views. Aakash owns the shell; Harshith owns catalog; Sathesh owns checkout/booking.	Exists; Vite + vanilla JS/CSS; currently mock-backed.
mobile/	Installable customer app, touch-first flow; decide if provider app is later required. Aakash coordinates shell conventions.	Create after team selects one mobile framework and supported devices.
backend/	Auth, roles, catalog, quote/order, service slots, seller, rider jobs and deployment. Pushpam owns backend; Vamsi owns persistence design.	Not present. Python FastAPI is the README proposal, not implemented code.
contracts/	Versioned OpenAPI, entity definitions, error format, status transitions and example payloads. Pushpam/Vamsi/Amrutha coordinate.	Create before feature integration.
docs/	Product decisions, setup, runbook, logistics decisions, release and API change notes.	Add project decisions here; review logistics/README.md selectively.

Technology decisions to record before foundation work

- Keep the existing Vite/vanilla JavaScript website as the starting point.
- Choose one cross-platform mobile framework on Day 1 after checking team skills, Android/iOS targets and build access. Flutter is mentioned in the README but no Flutter app exists; do not assume a scaffold is already available.
- FastAPI and PostgreSQL are the README architecture proposal. The logistics note mentions a different database approach. Pushpam and Vamsi should confirm the server/database choice and write it down before migrations begin.
- Start with REST/OpenAPI and server-side authorization. Add WebSockets or live location only when status polling and privacy requirements are understood.
- Put mock adapters behind the same API boundary for local UI work; never treat mock money, stock, fitment or slots as production truth.

Recommended default: one repository, separate web and mobile clients, one REST API, one relational database, one shared OpenAPI contract. The mobile framework and database are Day 1 decisions, not assumed completed implementations.

5. Core Workflows and Business Rules

Customer purchase and installation

Step	Customer or partner action	System response / evidence
1. Bike	Sign in and add bike make, model, year, variant and required attributes.	Validate required fields; use manual selection unless VIN lookup is verified.
2. Search	Search or filter batteries for the active bike.	Return compatible, incompatible, or unknown with reason and source; do not guess.
3. Select	Review specification, seller, stock, warranty and installation options.	Check service coverage and current slot availability.
4. Quote	Choose address and installation slot.	Server returns line items, estimate and total; client displays the result.
5. Confirm	Confirm order and any sandbox/mock payment step.	Revalidate stock/slot, prevent duplicate submission, persist order and appointment.
6. Fulfill	Follow the order and appointment.	Display timestamped transitions; reveal provider contact only after assignment.
7. Complete	Receive product and installation; raise support issue if needed.	Provider closes job with permitted evidence; customer sees completion.

Seller and provider handoff

Seller validates inventory and accepts or rejects the order with a reason. A ready order creates a delivery/service task. The scheduler or authorized operations role assigns a qualified provider. The provider accepts or declines, views only permitted customer and appointment details, and updates the job through allowed states. Customer notifications follow server events.

Order pricing and slot rules

Order total = battery subtotal + installation fee + delivery fee + approved taxes/charges - discounts. Currency, taxes, fee zones and exchange credits are team decisions; show no unapproved charge and no client-calculated authoritative total.

Slots need duration, capacity, buffer, local timezone, cutoff, hold expiry, cancellation and reschedule rules. An unpaid or abandoned checkout must not hold a slot forever. The server must prevent over-capacity bookings even when two customers confirm at the same time.

Status vocabulary to approve

Order / appointment milestone	Allowed actor / check
Pending payment -> Confirmed	Server payment result or explicitly approved non-payment order mode.
Seller accepted / rejected	Seller role for its own order; rejection reason required.
Preparing -> Ready for pickup	Seller role after inventory and readiness validation.
Assigned -> Accepted / Declined	Assigned provider only; decline reason and reassignment path.
On the way -> Installation in progress	Assigned provider only; timestamped and valid transition.
Completed / Cancelled / Failed	Authorized actor only; reason/evidence and customer notification as required.

6. Shared Data and API Contract

Publish one OpenAPI or equivalent contract in contracts/ before parallel feature integration. This is a proposed interface derived from the team guide; these endpoints are not implemented yet.

Entity	Minimum shared fields
Bike profile	bikeld, customerId, make, model, year, variant, optional VIN, required battery attributes, verification status.
Battery product	productId, sellerId, title, SKU, price, stock, voltage, capacity, chemistry, dimensions/connectors as needed, warranty, active status.
Fitment record	bike make/model/year/variant, constraints, compatible/incompatible result, evidence source, reviewer and last-reviewed date.

Entity	Minimum shared fields
Service offering	serviceId, providerId, coverage, duration, fee, capacity, hours, exclusions, cancellation rules.
Quote and order	orderId, customerId, bikeld, line items, subtotal, install/delivery fees, approved charges, discount, total, currency, payment/order state, address.
Appointment/job	appointmentId, orderId, providerId, slot start/end/timezone, address, assigned provider, status and timestamps.
Status event	eventId, entityId, previous/next state, actor role/id, timestamp, reason/note.

Initial REST boundaries

Area	Proposed endpoints
Catalog	GET /products?make=&model=&year= ; GET /products/{id} ; GET /bikes/{id}/compatible-products
Seller	POST/PUT /seller/products ; PATCH /seller/products/{id}/inventory ; GET /seller/orders ; PATCH /seller/orders/{id}
Service	GET /service-slots?providerId=&date=&address= ; POST /appointments
Order	POST /orders/quote ; POST /orders ; GET /orders/{id}
Rider	GET /rider/jobs ; POST /jobs/{id}/accept ; PATCH /jobs/{id}/status

Cross-module rules

Store timestamps in UTC and display in the appointment's local timezone. Use one money/currency representation. The server owns quote, stock, capacity, authorization and allowed transition checks. Use idempotency or duplicate protection for order/payment callbacks. Preserve status actor, time and reason for audit. Unknown compatibility remains unknown.

Example branch setup

```
git fetch origin
git switch -c feature/amrutha-rider origin/main
git push -u origin feature/amrutha-rider
```

Replace the example branch name with the unique name assigned to each person. Do not branch from the local stale main or from the logistics checkout.

7. Team Ownership and Day 1 Deliverables

Day 1 is for an accurate audit, decisions, boundaries and reviewable work. Every contributor should leave with one concrete artifact, a next ticket and a named blocker owner if something is unresolved. Planning and mock-backed work can proceed while API decisions are being made.

Time	Team activity	End evidence
09:30-10:00	Kickoff: product outcome, roles, current repo snapshot and scope owner.	Shared goal, owners and open decision list.
10:00-11:00	Inspect assigned area and mark implemented, mock-only, missing and risky.	Short module audit; no duplicate feature build.
11:00-12:00	Agree MVP boundaries, user journeys, roles, permissions and status vocabulary.	Scope/decision log and role/status table.
12:00-13:00	Agree website/mobile split and draft data/API contract.	Architecture note, screen map, contract owners and open fields.
14:00-15:00	Update branches from origin/main; register 12 personal branch names/reviewers.	Branch registry and PR/review convention.
15:00-16:00	Write Day 2-5 tickets with acceptance checks and dependencies.	Prioritized backlog with owner, evidence and blocked input.
16:00-16:30	Readout from each person: done, next, blocked.	Linked status updates and decision owners/deadlines.

Owner / branch	Day 1 work	Exact end-of-day delivery	Wait / coordinate
Harshitha / feature/harshitha-q	Frame one optional future optimization problem such as slot assignment. Keep it out of production checkout and fulfillment.	One-page memo: objective, input data, metric, classical baseline, feasibility and defer/continue recommendation.	Coordinate with Hema and Viswanath. Research can start now; MVP does not wait for it.

Owner / branch	Day 1 work	Exact end-of-day delivery	Wait / coordinate
Hema / feature/hema-q	Design a reproducible comparison for the selected research problem.	Benchmark outline with data, classical baseline, metric, reproducibility steps and stop/go criteria.	Pair with Harshitha; candidate problem is needed to finalize the benchmark. No core MVP dependency.
Viswanath / feature/viswanath-logistics	Draft coverage, delivery/install workflow, slot capacity/duration/buffer, fees, ETA, cutoff, cancellation and rescheduling.	Rules table with examples, edge cases, assumptions and named approver.	Coordinate with Sathwik, Vamsi and Pushpam. Draft now; approve core rules by Day 2.
Amrutha / feature/amrutha-rider	Own the cross-module checklist and map assigned jobs, accept/decline, job details, appointment/address, contact visibility and valid status actions.	Integration matrix (owner/API/field/dependency/blocker), rider workflow/status/contact notes, mock-only UI gaps, and a Day 2 integration ticket.	Start now. Real API integration waits for agreed order/appointment records, auth and status rules; the workflow and mock audit do not wait.
Aakash / feature/aakash-shell	Audit current shell/routes/responsive behavior; map website and mobile boundaries and choose the mobile framework decision path.	Platform note, screen map, responsive/device targets, shell tickets, mobile framework owner and open risks.	Needs team decision and Pushpam auth/API direction for final implementation; audit proceeds now.
Hima Bindu / feature/hima-quality	Create issue template and inspect the current customer/vendor/technician journeys at phone and desktop sizes.	Prioritized defects with reproduction, expected/actual, severity, owner, client and verification; initial release blockers.	Can audit web now. Mobile quality checks follow once the app build exists.

7. Team Ownership and Day 1 Deliverables (continued)

The remaining owners complete the same Day 1 loop: inspect what exists, state a deliverable, identify the handoff, and make the first ticket testable.

Owner / branch	Day 1 work	Exact end-of-day delivery	Wait / coordinate
Muni Sankar / feature/muni-catalog	Audit mock bike/battery/fitment data; define product fields and verified-fitment evidence, including compatible/incompatible/unknown.	Catalog dictionary, source/review workflow, five fitment/stock test cases, and unsupported mock claims flagged.	Coordinate with Harshith and Vamsi. Real fitment promises need an expert-reviewed data source.
Sathwik / feature/sathwik-billing	Audit current totals; specify authoritative quote items, tax/discount and payment states.	Quote contract and worked examples, payment scope decision, mock discrepancies; voice search stays optional.	Needs Viswanath fees and Pushpam API direction for final integration; audit starts now.
Vamsi / feature/vamsi-seller	Inventory product/seller/stock/slot/order fields; sketch persistence schema and seller actions.	Draft ER/schema, migration/seed plan, seller listing/inventory/order workflow and field owners.	Coordinate with Muni, Viswanath and Pushpam; design starts now, final schema depends on shared fields.
Harshith / feature/harshith-catalog-ui	Review search/detail and define loading, empty, error, unknown and out-of-stock behavior for both clients.	Web/mobile screen matrix, required API fields, prototype gaps and observable acceptance checks.	Needs Muni fitment fields/source and Aakash conventions to integrate; planning/mock UI can proceed.
Pushpam / feature/pushpam-backend	Confirm no server exists in the inspected branches; plan auth/roles, API conventions, environments, logs and deployment.	Backend decision note: proposed stack, authorization, error/version rules, configuration and first skeleton/setup ticket.	Needs team stack and Vamsi schema direction; architecture work starts now.
Sathesh / feature/sathesh-checkout	Audit booking, totals, slot UI, order status and provider reveal; map matching customer flow on both clients.	Screen/flow matrix, quote/order/appointment fields, payment/slot conflict paths and mock behavior called out.	Needs Sathwik, Viswanath, Pushpam and Amrutha contracts for integration; audit and screen plan proceed.

Day 1 team packet

By end of day, publish: an MVP scope/decision log; customer/seller/provider/admin flow and permission table; website/mobile architecture decision; contract owner and draft fields; 12 unique personal branches; 12 owned next tickets; and completed/next/blocked updates. An open decision is acceptable only when an owner and due date are recorded.

8. Branch, Review, and Integration Rules

Every contributor must have a separate personal branch. One owner per branch and deliverable. Keep feature work off main; open a reviewable pull request into the agreed integration branch. A branch may contain web and mobile changes when they are one coherent feature owned by the same contributor.

Rule	Team practice
Base	Fetch latest origin/main. The local main is behind and the current checkout is logistics; do not use either stale local ref as a base.
Existing branches	ui acd4855 is already in main. Review logistics/README.md as a separate contribution; do not merge the full old logistics tree without checking its UI files.
Naming	Use unique feature/<owner>-<area> names. Register owner, module, reviewer, base commit and client(s) in the branch sheet.
Commits	Small, focused changes. Avoid mixing schema, UI polish and unrelated documentation in one pull request.
Pull request	Include purpose, acceptance evidence, screenshots/demo for UI, contract/schema changes, migration notes and risks.
Review	At least one adjacent module owner reviews contract and behavior. Author resolves comments and updates their own branch.
Integration	Merge small increments after agreed contracts. Rebase or merge latest base regularly; never force-push shared integration branch.
Mock data	Clearly label fixtures and keep them behind a replaceable adapter. No mock quote, stock or fitment result is a production guarantee.

Suggested individual branches

Contributor	Branch	Contributor	Branch
Harshitha	feature/harshitha-q	Hema	feature/hema-q
Viswanath	feature/viswanath-logistics	Amrutha	feature/amrutha-rider
Aakash	feature/aakash-shell	Hima Bindu	feature/hima-quality
Muni Sankar	feature/muni-catalog	Sathwik	feature/sathwik-billing
Vamsi	feature/vamsi-seller	Harshith	feature/harshith-catalog-ui
Pushpam	feature/pushpam-backend	Sathesh	feature/sathesh-checkout

These are proposed names, not branches that already exist. Create each from the same agreed current main base after a fetch.

9. Eight-Week Delivery Roadmap

The eight-week schedule is a baseline. Each phase closes on evidence. If a contract, environment, verified fitment source, payment sandbox or app framework is late, surface the dependency and re-estimate rather than silently slipping integration.

Time	Primary owners	Work and end-of-phase evidence
Week 1 - scope and design	All; Amrutha tracks integration; product owner decides scope.	Confirm included/deferred scope, journeys, bike fields/fitment source, prices/slots/status rules, website/mobile framework, stack, reviewers and first contract draft. Exit: approved backlog, decision log, branch registry and API/data owners.
Week 2 - foundation	Pushpam, Vamsi, Aakash, mobile owner.	Create backend/app skeletons, auth/roles, schema/migrations, API conventions, mock adapter boundary, dev setup and first deploy. Exit: website, mobile shell and backend run with test roles.
Weeks 3-4 - catalog and seller	Muni, Vamsi, Pushpam, Harshith, Aakash/mobile owner.	Structured battery/fitment data, search/detail on both clients, seller listing/stock/order response. Exit: seller data reaches catalog; customer sees verified/unknown results.
Weeks 4-5 - quote and booking	Sathwik, Viswanath, Pushpam, Vamsi, Sathesh, client contributors.	Server quote, line items, payment test mode, service coverage/slots, appointment/order creation, confirmation. Exit: test order with valid installation slot and exact total.
Weeks 5-6 - rider fulfillment	Amrutha, Viswanath, Pushpam, Sathesh, Vamsi.	Job/appointment assignment, accept/decline, authorized provider details, status events, customer updates. Exit: assigned provider updates a job and customer sees the permitted update/contact.
Week 7 - integration/hardening	All; Hima coordinates defects.	Role checks, empty/error/offline states, stock/slot conflicts, cancellation/reschedule, phone and desktop review. Exit: end-to-end staging journey with no release blocker.
Week 8 - release readiness	Pushpam leads; all feature owners.	Release candidate, deployment and rollback notes, logs/config/backup owner, demo accounts, known issues and final acceptance evidence.

First five working days

Day	Team output
1	Scope, branch registry, repo audit, platform decision owner, first module artifacts and blockers.
2	Customer/seller/provider journeys, role permissions, status transitions, service/fee/slot rules.
3	Schema/API payloads, error format, IDs, money units, timezone and authorization contract.
4	App/backend skeleton, roles, sample catalog/test users, developer setup and first environment.
5	Demo bike profile -> compatible/unknown outcomes -> battery detail. Log blockers and re-plan.

10. Dependencies and When to Wait

Do not idle because an integration is blocked. Continue an independent audit, design, mock fixture, acceptance test or contract draft; label the exact production integration step that must wait.

Work	Can start now	Must be agreed before production integration	Owners
Product scope and journeys	Yes: audit and draft flows.	Scope decision owner and included/deferred list.	Product lead; all; Hima tracks.
Catalog / fitment	Audit current fixtures and create UI states.	Structured fields, evidence source/reviewer, exact/unknown/incompatible rules.	Muni, Harshith, Vamsi, Pushpam.
Quote / booking	Audit UI and totals; write examples.	Fee/tax/currency rules, server quote, slot capacity/hold/conflict behavior.	Sathwik, Viswanath, Sathesh, Pushpam.
Backend / schema	Architecture, ER/API draft, setup ticket.	Database/stack choice, IDs, money/time/errors, auth and migrations.	Pushpam, Vamsi.
Rider panel	Workflow map, mock jobs, contact/status matrix.	Persisted order/appointment/job contract and server permissions/transitions.	Amrutha, Pushpam, Viswanath, Vamsi.
Mobile app	Screen map and framework evaluation.	One framework, supported OS/device scope, signing/build access and API contract.	Aakash and mobile contributors.
GPS / voice / quantum	Privacy/scope decision or isolated research only.	Consent/reliability, stable search/fitment, measured value and product approval.	Amrutha/Pushpam; Sathwik; Harshitha/Hema.

Escalation rule: if the same dependency blocks more than one work session, post the exact decision or input needed, its owner, affected task and date needed. Keep moving on work that does not depend on it.

Decision owners and target dates

Decision	Owner	Target
MVP scope authority and brand name (Riderzpro repo / VoltFit web identity)	Product lead to name; Aakash records.	Day 1
Mobile framework and supported devices	Aakash plus team lead.	Day 1
VIN lookup provider or manual-only	Muni plus product reviewer.	Day 1
Payment provider/test mode and currency	Sathwik and Pushpam.	Day 1
Service coverage, fees, capacity and cancellation	Viswanath.	Draft Day 1; approve Day 2
Database and backend stack	Pushpam and Vamsi.	Day 2
Shared contract location/version and status permissions	Pushpam, Vamsi, Amrutha.	Owner Day 1; draft Day 3

11. Quality, Testing, and Definition of Done

A feature is done when its success and failure behaviors work, authorization is checked, the feature owner records evidence, and the change is documented. Hima coordinates the defect board; each module owner verifies their own work.

Scenario	Acceptance result
Compatible purchase	Customer creates a bike profile, sees a verified in-stock match, reviews details, books installation, sees full total, confirms and receives an order.
Unknown / incompatible	System distinguishes unknown from incompatible, explains next action and never claims an unsupported fit.
Stock change	Server prevents confirmation when product becomes unavailable and returns a useful recovery path.
Slot conflict	Capacity cannot be double-booked; the losing request gets refreshed availability.
Price consistency	Quote equals saved order; price/fee changes are shown for confirmation first.
Seller response	Seller can access only its orders and accept/reject with reason; customer sees the event.
Provider assignment	Assigned provider can see the job; customer sees provider contact only after assignment.
Status update	Authorized provider can make valid transitions; invalid transitions are rejected and audited.
Role isolation	Customer, seller, provider, rider and admin cannot access unauthorized records/actions.
Responsive parity	Core journey works on supported phone sizes and desktop without clipped actions or hidden totals.

Test layers

Layer	Coverage
Unit	Fitment decisions, quote arithmetic, valid state transitions, fee/slot rules, authorization and validation.
API/integration	Catalog -> quote -> order -> appointment -> assignment -> status event; persistence and ownership checks.
Concurrency	Two customers competing for last stock or slot; duplicate order/payment callback; idempotent retry.
Client	Web and app loading/empty/error/offline/retry states, form validation, accessible labels and responsive layout.
End-to-end	Seeded compatible, unknown, incompatible, out-of-stock, no-slot, seller rejection, provider decline, cancellation and completion journeys.

Definition of done for each pull request

The PR has one purpose and owner; acceptance criteria are observable; mock versus real behavior is identified; authorization and error paths are considered; web/mobile parity impact is named; migrations and contract changes are documented; and a reviewer can reproduce the evidence.

12. Security, Privacy, and Operational Requirements

Area	Minimum MVP requirement
Identity and roles	Authentication and record ownership enforced by backend on every request. UI role switching is development-only.
Customer/provider data	Limit contact/address access to the assigned parties and approved support roles. Avoid logging secrets or unnecessary personal data.
Location	Use status and timestamp updates for MVP. Collect live location only after privacy, consent, retention and support decisions.
Fitment evidence	Store source and last-reviewed date. Changes have a named reviewer; missing evidence is unknown.
Price and payments	Server quote is authoritative; secrets stay server-side; payment callbacks are validated and duplicate-safe.
Audit	Record actor, timestamp, prior/new status and reason for consequential status changes.
Reliability	Handle duplicate submits, changed stock, slot conflicts, provider declines and network retries without losing order state.
Time and money	Store timestamps in UTC; show appointment-local time; use a consistent decimal/minor-unit money representation and currency.

Project risks and response

Risk	Impact	Response / owner
False fitment result	Unsafe purchase and lost customer trust.	Start with a small reviewed catalog; unknown stays unknown. Muni + product reviewer.
Branch divergence	UI work or contract decisions are lost or overwrite newer code.	Use origin/main as base; UI is already merged; port logistics requirements selectively. Maintainer.
Unclear fees or slot rules	Conflicting totals, bad booking or double booking.	Viswanath owns rules; Sathwik quote; server owns final total and slot capacity.
Late API integration	Separate modules do not fit at demo time.	Publish contract by Day 3; use labeled fixtures; Amrutha tracks interface dependencies.
Web/mobile duplication	Different fitment, price or status behavior.	One API/data contract and parity cases; one mobile framework decision. Aakash + Pushpam.
Scope expansion	Core customer journey slips behind quantum, GPS or broad logistics work.	Keep optional work isolated; require product approval and record displaced scope.
Unowned defects	High-risk issues reach release.	Hima maintains owner/severity/repro/verification; feature owners fix and verify.

Operating rhythm

Each contributor posts a short completed / next / blocked update twice a week with a branch, PR, demo or artifact link. Review work in small increments. Weekly planning selects a limited set of outcomes, owners, dependencies and acceptance checks. Demonstrations should show behavior, error handling and role checks rather than screenshots alone.

13. Release Gate and End-to-End Demo

Do not call the MVP ready because the screens look complete. The release candidate must demonstrate the integrated customer and partner workflow against persistent backend data, with roles and business rules enforced.

Release check	Pass condition
Customer entry	Customer signs in and creates a bike profile with required fields.
Catalog trust	Products show specification, price, seller, stock and clear compatibility outcome/source.
Quote and booking	Customer books a valid installation slot; full server total is shown before confirmation and matches the saved order.
Seller operations	Seller maintains listing/stock and responds only to its own order.
Provider workflow	Rider/technician sees assigned job, accepts/declines, updates permitted states and records completion evidence.
Customer tracking	Customer sees status and timestamps, and provider contact only after assignment.
Security	Backend enforces role and record authorization; invalid transitions are rejected.
Web/mobile	Core customer journey reviewed on responsive website and supported mobile devices.
Defects	No open blocker or high-severity defect; known issues have workarounds and owners.
Operations	Staging, demo accounts, logs, configuration, migration, backup/recovery and rollback notes are ready.

Final demo script

1. Create or select a bike profile.
2. Search and show a verified compatible battery, then demonstrate an unknown or incompatible outcome.
3. Open product details and show seller, stock, specifications and warranty.
4. Request quote; review battery, install, delivery, tax/approved charges, discount and total.
5. Select an available slot, confirm the order and show the saved order summary.
6. Seller accepts or rejects; provider assignment is made with correct permissions.
7. Provider updates a valid status; customer sees timestamp and eligible contact.
8. Attempt a slot/stock conflict and an unauthorized status change; show safe rejection and recovery.

Completion standard: both clients use the same verified catalog, server quote, order, appointment and status records; the core journey passes on web and mobile; access is enforced; and the team can deploy, observe, support and recover the release.